

IBM DataPower Orchestrator - Discovery Feature

NEW FEATURE | Automatically discover all domains and certificate stores across your DataPower appliance. No more manually creating hundreds of cert store definitions.

Before vs. After Discovery

Without Discovery (Manual Setup)

- Admin must know every domain name on the appliance
- Each domain + store combination created by hand in Keyfactor Command
- 50 domains x 2 stores = 100 cert store definitions manually configured
- New domains added to DataPower require manual store creation
- Environments (prod, test, dev, sandbox) multiply the effort
- Human error risk - missed domains mean unmanaged certificates

With Discovery (Automated)

- Point Discovery at the appliance - it finds all domains automatically
- All certificate store directories (cert, pubcert, sharedcert) detected per domain
- Store paths returned in ready-to-use format: domain\store
- New domains picked up on next scheduled Discovery run
- Run once per environment to discover everything
- Complete coverage - no domains or stores overlooked

Customer Scenario

A typical enterprise DataPower deployment with multiple environments, each containing dozens of domains:

Environment	Impact
Production	50 domains x 2 cert stores = 100 store definitions auto-discovered in one job
Test	40 domains x 2 cert stores = 80 store definitions auto-discovered in one job
Dev	30 domains x 2 cert stores = 60 store definitions auto-discovered in one job
Sandbox	20 domains x 2 cert stores = 40 store definitions auto-discovered in one job
Total	280 cert store definitions - discovered automatically with 4 Discovery jobs

How Discovery Works

Step 1: Keyfactor Triggers Discovery Job

Keyfactor Command schedules a Discovery job targeting the DataPower appliance. Only the appliance hostname/IP and credentials are needed - no domain or store path required.

Step 2: Enumerate All Domains

The orchestrator calls `GET /mgmt/domains/config/` on the DataPower REST Management Interface. This returns every application domain configured on the appliance.

Step 3: Discover Certificate Stores Per Domain

For each domain found, the orchestrator calls `GET /mgmt/filestore/{domain}` to list the top-level filestore directories. It filters for certificate-relevant directories: `cert`, `pubcert`, and `sharedcert`.

Step 4: Build Store Paths

Each domain + directory combination is formatted as a store path using the existing convention: `domain\directory` (e.g., `production-api\cert`).

Step 5: Submit to Keyfactor Command

All discovered store paths are submitted back to Keyfactor via the `SubmitDiscoveryUpdate` callback. Keyfactor Command can then auto-create the corresponding certificate store definitions.

Resilient by design: If the orchestrator cannot access a specific domain's filestore (e.g., due to permissions), it logs a warning and continues discovering the remaining domains. One inaccessible domain does not block the entire job.

Store Path Format

All operations in the DataPower Orchestrator (Discovery, Inventory, Add, Remove) use a consistent store path format:

```
<domain>\<directory>
```

Certificate Store Directories

Directory	Scope	Contents
cert	Per-domain	Domain-specific certificates and private keys (CryptoCertificate/CryptoKey objects)
pubcert	Appliance-wide	Public/trusted certificates shared across all domains
sharedcert	Appliance-wide	Shared certificates that persist across firmware upgrades

Examples

Store Path	Description
default\pubcert	Public certificate store in the default domain
production-api\cert	Private key certificates in the production-api domain
staging\pubcert	Public certificates in the staging domain
testdomain\sharedcert	Shared certificates in the testdomain domain

Example Discovery Output

For a DataPower appliance with 4 domains, Discovery returns store paths like:

```
default\cert
default\pubcert
production-api\cert
production-api\pubcert
staging-api\cert
staging-api\pubcert
internal-services\cert
internal-services\pubcert
```

Each of these becomes a certificate store in Keyfactor Command, ready for Inventory and Management operations.

DataPower API Calls Used

GET /mgmt/domains/config/

Returns all application domains configured on the DataPower appliance.

```
{
  "domain": [
    { "name": "default" },
    { "name": "production-api" },
    { "name": "staging-api" },
    { "name": "internal-services" }
  ]
}
```

GET /mgmt/filestore/{domain}

Returns the top-level filestore directories for a specific domain. The orchestrator filters for cert, pubcert, and sharedcert.

```
{
  "filestore": {
    "directory": [
      { "name": "cert" },
      { "name": "chkpoints" },
      { "name": "config" },
      { "name": "local" },
      { "name": "pubcert" },
      { "name": "sharedcert" }
    ]
  }
}
```

DataPower JSON quirk: When only a single item exists (e.g., one domain or one directory), DataPower returns a plain object instead of a single-element array. The orchestrator handles this automatically, consistent with how the existing Inventory job handles the same behavior.

Implementation Architecture

New Files

File	Description
Jobs/Discovery.cs	Main Discovery job class. Implements IDiscoveryJobExtension. Orchestrates domain enumeration and store path collection.
Models/Requests/ListDomainsRequest.cs	HTTP request model for GET /mgmt/domains/config/
Models/Requests/ListFileStoreRequest.cs	HTTP request model for GET /mgmt/filestore/{domain}
Models/Responses/ListDomainsResponse.cs	Response deserialization for domain listing. Includes single-item variant for DataPower JSON quirk.
Models/Responses/ListFileStoreResponse.cs	Response deserialization for filestore directory listing.
Models/SupportingObjects/DomainInfo.cs	Domain name and href properties from the DataPower domains API.
Models/SupportingObjects/FileStoreDirectory.cs	Directory name and href properties from the filestore API.

Modified Files

File	Description
Client/DataPowerClient.cs	Added ListDomains() and ListFileStoreDirectories() methods with single-item JSON quirk handling.
manifest.json	Registered CertStores.DataPower.Discovery job extension type.
integration-manifest.json	Set supportsDiscovery: true and Discovery: true in supported operations.